JAVA CORE CONCEPTS

Basic Maths for DSA

Digit extraction, number manipulation, prime checking, and Euclidean algorithms in Java

Basic Maths for programming is the critical bridge between **syntax learning** and **logical problem solving**. Rather than complex theoretical proofs, this topic focuses on digit manipulation, modulo arithmetic, condition building, loop boundaries, and algorithmic optimizations essential for Data Structures & Algorithms (DSA).

1. Number Basics & Arithmetic Operations

In beginner DSA problems, we primarily work with **integers** (whole numbers like 5, 18, 120) and **decimal numbers** (floating-point numbers like 3.14, 87.5).

Operation	Symbol	Example (a = 10, b = 3)	Result
Addition	`+`	`a + b`	13
Subtraction	`-`	`a - b`	7
Multiplication	`*`	`a * b`	30
Division	`/`	`a / b`	3 (Integer Division)
Modulo (Remainder)	`%`	`a % b`	1

⚠️ **Integer Division & Modulo Rules**

• When operating on two integers, `10 / 3` yields `3` (the fractional part `0.333...` is truncated).
• Modulo `10 % 3` yields `1` because $10 = (3 imes 3) + 1$. Modulo is fundamental for digit extraction and circular indexing.

2. Core Digit Extraction Mechanics

Working with individual digits of a number relies on two primary operations:

- **Extract Last Digit:** `n % 10` (Returns the rightmost digit).
- **Remove Last Digit:** `n = n / 10` (Shifts the number right by removing the rightmost digit).

Example Matrix (for `n = 1234`):

- `1234 % 10` → `4` (Extracted digit)
- `1234 / 10` → `123` (Remaining number)

3. Digit Problems (Count, Sum, Reverse)

Problem 1: Count Digits in a Number

```java
public class Main {
    public static void main(String[] args) {
        int n = 12345;
        int count = 0;
        while (n > 0) {
            count++;
            n = n / 10;
        }
        System.out.println("Number of digits = " + count);
    }
}
```

Problem 2: Sum of Digits

```java
public class Main {
    public static void main(String[] args) {
        int n = 1234;
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum = sum + digit;
            n = n / 10;
        }
        System.out.println("Sum of digits = " + sum);
    }
}
```

Problem 3: Reverse a Number

```java
public class Main {
    public static void main(String[] args) {
        int n = 1234;
        int rev = 0;
        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit; // Shifts existing digits left by 1 place
            n = n / 10;
        }
        System.out.println("Reversed number = " + rev);
    }
}
```

4. Number Classification & Properties

Problem 4: Palindrome Number Check

```java
public class Main {
    public static void main(String[] args) {
        int n = 121;
        int original = n;
        int rev = 0;
        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n = n / 10;
        }
        if (original == rev) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

Problem 5: Prime Number (Optimized to $\sqrt{N}$)

Factors of a number always occur in pairs (e.g., for 36: $1 imes 36$, $2 imes 18$, $3 imes 12$, $4 imes 9$, $6 imes 6$). Therefore, checking up to $\sqrt{N}$ ($i imes i \le N$) is sufficient and optimized from $O(N)$ to $O(\sqrt{N})$.

```java
public class Main {
    public static void main(String[] args) {
        int n = 7;
        boolean isPrime = true;

        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i * i <= n; i++) { // Optimized loop up to sqrt(n)
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        System.out.println(isPrime ? "Prime" : "Not Prime");
    }
}
```

Problem 6: Factorial of a Number

```java
public class Main {
    public static void main(String[] args) {
        int n = 5;
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial = " + fact);
    }
}
```

5. GCD & LCM Algorithms

Problem 7 & 8: GCD (Euclidean Algorithm) & LCM

The **Euclidean Algorithm** states that $GCD(a, b) = GCD(b, a \pmod b)$ until $b = 0$. Using the identity $LCM imes GCD = a imes b$, we compute $LCM = rac{a imes b}{GCD}$.

```java
public class Main {
    public static void main(String[] args) {
        int a = 12, b = 18;
        int x = a, y = b;

        // Euclidean GCD Computation
        while (y != 0) {
            int temp = y;
            y = x % y;
            x = temp;
        }
        int gcd = x;
        int lcm = (a * b) / gcd;

        System.out.println("GCD = " + gcd); // 6
        System.out.println("LCM = " + lcm); // 36
    }
}
```

6. Additional Core Maths Problems

Problem 9: Armstrong Number

```java
public class Main {
    public static void main(String[] args) {
        int n = 153;
        int original = n;
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum += (digit * digit * digit);
            n /= 10;
        }
        System.out.println(sum == original ? "Armstrong Number" : "Not Armstrong Number");
    }
}
```

Problem 10: Power Calculation (a^b)

```java
public class Main {
    public static void main(String[] args) {
        int a = 2, b = 5;
        int ans = 1;
        for (int i = 1; i <= b; i++) {
            ans *= a;
        }
        System.out.println("Power = " + ans); // 32
    }
}
```

Problem 11: Perfect Number Check

```java
public class Main {
    public static void main(String[] args) {
        int n = 6;
        int sum = 0;
        for (int i = 1; i < n; i++) {
            if (n % i == 0) {
                sum += i;
            }
        }
        System.out.println(sum == n ? "Perfect Number" : "Not Perfect Number");
    }
}
```

Problem 12: Count Even Digits

```java
public class Main {
    public static void main(String[] args) {
        int n = 248531;
        int count = 0;
        while (n > 0) {
            int digit = n % 10;
            if (digit % 2 == 0) count++;
            n /= 10;
        }
        System.out.println("Even digits count = " + count);
    }
}
```

Problem 13: Print Primes in Range 1 to N

```java
public class Main {
    public static void main(String[] args) {
        int n = 20;
        for (int num = 2; num <= n; num++) {
            boolean isPrime = true;
            for (int i = 2; i * i <= num; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
            if (isPrime) System.out.print(num + " ");
        }
    }
}
```

7. Key Formula Matrix & Pitfalls

Concept / Rule	Java Implementation	Key Insight / Application
Extract Last Digit	`n % 10`	Gets rightmost digit
Remove Last Digit	`n / 10`	Shifts number right
Even / Odd Check	`n % 2 == 0`	Evaluates parity
Reverse Building	`rev = rev * 10 + digit`	Shifts left and appends digit
Euclidean GCD	`GCD(a, b) = GCD(b, a % b)`	Loops until $b = 0$
LCM Formula	`LCM = (a * b) / GCD`	Derived via $LCM imes GCD = a imes b$

⚠️ Common Beginner Mistakes

- **Original Value Destruction:** Modifying `n` directly in a loop without preserving `int original = n;` beforehand.
- **Missing Loop Updates:** Forgetting `n = n / 10` inside a `while(n > 0)` loop, causing an infinite loop.
- **Integer Truncation Ignorance:** Forgetting that `5 / 2` equals `2` in Java integer arithmetic.